AUTONATEAI SCREEPS LEAGUE

Tutorial 12: PvP Fundamentals

A practical local-server Screeps module **docs/tutorials/12-pvp-fundamentals.md**

Episode 12: The First Skirmish

"Scouting tells you where the danger is before it arrives," your collaborator says. "It doesn't tell you what to do once it's here. That's combat, and none of these nine roles were built for it – you've basically got a whole company with no security team and a genuinely great CRM."

Every creep you've written so far either gathers, builds, or moves energy. None of them can throw a punch.

Goal

Give the colony its first combat-capable creep and understand the mechanics well enough to use it correctly:

- Learn what `ATTACK`, `RANGED_ATTACK`, `HEAL`, and `TOUGH` actually do, in numbers
- Understand rampart tanking – why a defender should fight from a rampart tile, not in open terrain
- Add `role.defender.js` and a threat-responsive population override
- Trigger a real test invasion on this local server and watch the defender fight it

Prerequisites

Tutorial 11 is complete:

- `role.scout.js` is patrolling and `Memory.rooms` has real intel.
- The tower and rampart infrastructure from Episode 7 exists and is being maintained.

Step 1: The Numbers

Every combat body part has a fixed, non-negotiable effect per tick:

Part	Action	Effect
`ATTACK`	`creep.attack(target)`	30 damage, melee range (1 tile)
`RANGED_ATTACK`	`creep.rangedAttack(target)`	10 damage, range 3
`HEAL`	`creep.heal(target)`	12 hp restored, melee range
`HEAL`	`creep.rangedHeal(target)`	4 hp restored, range 3
`TOUGH`	none	No action. Adds 100 hp of buffer per part. Its only job is to die last.

`ATTACK` hits harder than `RANGED_ATTACK` per part, but only at range 1 – meaning a melee attacker has to close distance while taking ranged damage the whole way in. There's no part that's simply "better"; the tradeoff is the mechanic.

Step 2: Understand Rampart Tanking

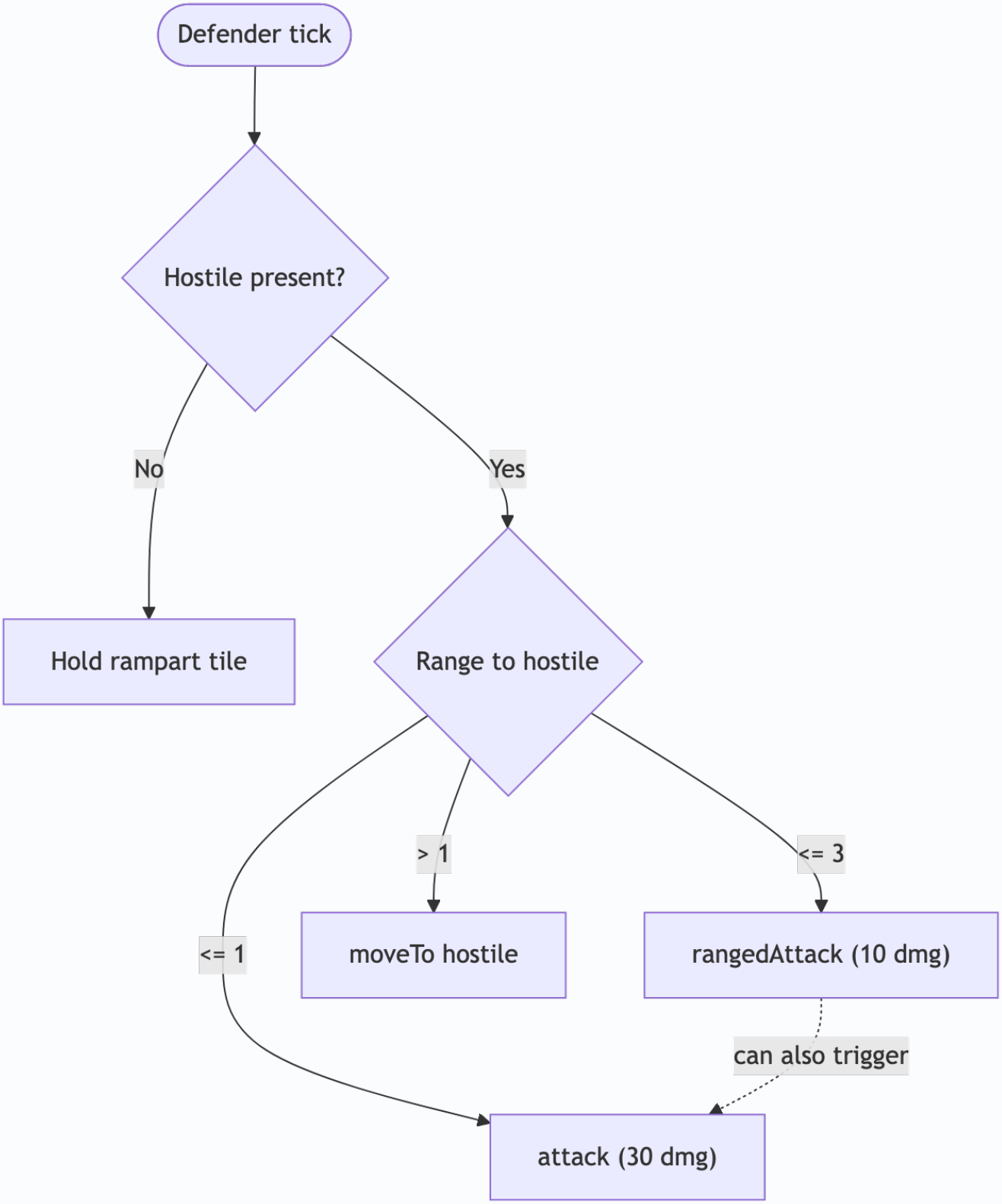
A rampart isn't just a wall. A creep standing *on top of* your own rampart is protected by it – an enemy outside cannot damage that creep directly until the rampart's own hit points are reduced to zero first. The rampart absorbs the hit; the creep standing on it doesn't take damage from outside attacks at all.

This is why Episode 7 had the builder repairing ramparts instead of leaving them at their low starting hit points. A rampart with a handful of hit points protects nothing – a single hostile hit clears it in one tick. A rampart with real hit points behind it is what makes "stand your ground" a viable defensive tactic instead of a way to lose a creep for nothing.

Step 3: Add `role.defender.js`

```
const roleDefender = {
  run(creep) {
    const hostile = creep.room.find(FIND_HOSTILE_CREEPS)[0];

    if (!hostile) {
      const rampart = creep.room.find(FIND_MY_STRUCTURES, { filter: { structureType:
STRUCTURE_RAMPART } })[0];
      if (rampart && !creep.pos.isEqualTo(rampart.pos)) {
        creep.moveTo(rampart, { visualizePathStyle: { stroke: '#ff0000' } });
      }
      return;
    }

    const range = creep.pos.getRangeTo(hostile);

    if (creep.getActiveBodyparts(RANGED_ATTACK) > 0 && range <= 3) {
      creep.rangedAttack(hostile);
    }
    if (creep.getActiveBodyparts(ATTACK) > 0 && range <= 1) {
      creep.attack(hostile);
    }
    if (range > 1) {
      creep.moveTo(hostile, { visualizePathStyle: { stroke: '#ff0000' } });
    }
  },
};

module.exports = roleDefender;
```

With no hostile present, the defender parks itself on a rampart tile and waits. That's deliberate — a defender standing in open terrain is just a creep waiting to be a bad trade.



`Ranged` and `Melee` aren't mutually exclusive branches – both checks run independently, so a creep at range 1 with both parts fires both in the same tick.

Step 4: Give It a Body and a Threat-Responsive Population

```
const roleDefender = require('role.defender');

const ROLE_BODIES = {
  // ...existing roles from Episode 8...
  defender: [TOUGH, TOUGH, ATTACK, ATTACK, RANGED_ATTACK, MOVE, MOVE, MOVE, MOVE],
};

const POPULATION = {
  // ...existing roles from Episode 8...
  defender: 0,
};

function getPopulationTargets(room) {
  const hostileCount = room.find(FIND_HOSTILE_CREEPS).length;
  return Object.assign({}, POPULATION, {
    defender: hostileCount > 0 ? 2 : 0,
  });
}

function spawnMissingRoles() {
  const spawn = Game.spawns.Spawn1;
  if (spawn.spawning) {
    return;
  }

  const targets = getPopulationTargets(spawn.room);
  for (const role in targets) {
    const currentCount = _.filter(Game.creeps, (creep) => creep.memory.role === role).length;
    if (currentCount < targets[role]) {
      const name = `${role}${Game.time}`;
      spawn.spawnCreep(ROLE_BODIES[role], name, { memory: { role } });
      break;
    }
  }
}
```

`defender` sits at a population target of `0` under normal conditions – the colony doesn't waste energy maintaining a standing army against nothing. The moment `FIND_HOSTILE_CREEPS` returns anything in the home room, the target jumps to `2` and `spawnMissingRoles` starts producing defenders on the very next spawn cycle.

Add the matching `else if` branch to the dispatch loop for `defender`, same as every role before it.

Step 5: Trigger a Test Invasion

Use the sparring-ground endpoint from Episode 7, Step 7:

```
curl -X POST http://localhost:21025/local/api/sparring/wave \
  -H 'Content-Type: application/json' \
  -d '{"room": "<your-room-name>"}'
```

This time watch `role.defender.js` instead of the tower. The population override from Step 4 should notice `FIND_HOSTILE_CREEPS` returning something and start producing defenders on the next spawn cycle – if you don't already have two on hand, there will be a gap between the invader arriving and your first defender reaching it. That gap is real and worth observing, not a bug to fix in this episode.

Checkpoint:

```
_.filter(Game.creeps, (creep) => creep.memory.role === 'defender').length
```

Expected result: `2`, once `spawnMissingRoles` catches up to the threat.

Watch the client as the defender closes distance. It should fire `rangedAttack` starting at range 3, keep firing as it closes, and add `attack` once it's adjacent — both actions can land in the same tick, which is why Step 3's code checks both conditions independently instead of using `else if`.

If you built ramparts with real hit points in Episode 7, try positioning a defender on one before the invader arrives (its default idle behavior already does this) and compare how the fight goes versus a defender caught in the open.

Because the endpoint is just a `curl` call, nothing stops you from running it again once the first invader is dealt with — call it a second and third time and watch whether your defender population keeps pace, or whether two defenders start struggling against back-to-back waves. That repeatability is the whole point, and Episode 13 turns it into an actual practice loop.

> 📸 **Screenshot placeholder:** A defender and an invader mid-fight, close enough to show both creeps' health bars dropping — the clearest single image of the numbers from Step 1 actually applying in real time.

If TooAngel or another NPC bot is placed nearby (`docs/local-screeps.md`), it may eventually send creeps at your room on its own — that's not a scripted wave, it's a real opponent's actual decision. Treat any contact from it as a bonus data point, not something to force on a schedule.

Troubleshooting

If the defender never attacks despite a hostile being present, check `creep.getActiveBodyparts(ATTACK)` and `creep.getActiveBodyparts(RANGED_ATTACK)` directly — a body part reduced to 0 hp by damage stops counting as "active" even though it's still listed in `creep.body`.

If `spawnMissingRoles` never produces a defender during a threat, confirm `getPopulationTargets` is actually being called instead of the raw `POPULATION` object — this is an easy line to miss when copying the Episode 8 version forward.

If the defender walks into the open instead of holding a rampart tile, confirm at least one rampart exists and has been assigned real hit points by the builder (Episode 7, Step 6) — `find(FIND_MY_STRUCTURES, ...)` will still return a rampart with 1 hit point, and standing on it won't help much.

If the wave endpoint doesn't produce an invader, see Episode 7's troubleshooting — the client's Invasion panel is the guaranteed fallback.

Completion Criteria

You are done when:

- `role.defender.js` exists and is wired into `main`.
- The population system spawns defenders only when a hostile is present in the home room.
- You can explain, from the Step 1 table, why a defender body mixes `TOUGH`, `ATTACK`, and `RANGED_ATTACK` instead of maximizing just one.
- You've triggered a test invasion and watched a defender actually engage it, not just verified the code compiles.

Learning Notes

After completing the tutorial, write down:

- Why does the defender check `RANGED_ATTACK` before `ATTACK` in the same tick? What would go wrong if a creep with both tried to only ever melee?
- What's the failure mode if `defender` population jumps to `2` but the room's `energyCapacityAvailable` can't afford even one defender body?
- How many ticks passed between the invader spawning and your first defender reaching it? What would close that gap — a standing defender population instead of `0`, or something else?
- Boosts (a lab structure, unlocked at RCL6) can amplify these numbers significantly. Why does this episode not cover them?

- If a hostile appeared in `Memory.remoteRoom` instead of your home room, would anything in this episode's code respond? Should it?

Next: Episode 13 — The Sparring Ground

One test invader proved the defender works. It didn't prove the defender is *good* — that takes losing to something a few dozen times and adjusting.

"A single wave is a smoke test, not a training loop," your collaborator says. "You've got a `curl` command that can throw another one at you any time you want. Let's actually use it that way — call it load testing, except the load is trying to kill your workers."

See `docs/roadmap.md` for the full season.